

UNIDAD 9 (parte 2):

PERSISTENCIA - ACCESO A BASES DE DATOS

**Programación
CFGS DAW**

Adaptado y actualizado por: José Miguel Blázquez
Autores: Carlos Cacho, Raquel Torres y Lionel Tarazón

2023/2024

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDO

1. Introducción.....	4
2. Repaso del lenguaje SQL.....	4
2.1 Comandos.....	5
2.2 Cláusulas.....	5
2.3 Operadores.....	5
2.4 Ejemplos de sentencias SQL.....	6
3. JDBC.....	8
3.1 Drivers JDBC.....	9
4. Acceso a bases de datos mediante interfaz gráfica (NetBeans).....	9
4.1 Instalación del JDBC Driver.....	9
4.2 Conexión a Base de Datos mediante el JDBC Driver.....	11
4.3 Crear una tabla.....	13
4.3.1 Mediante sentencias SQL.....	13
4.3.2 Mediante el asistente de creación de tablas.....	13
4.4 Insertar datos.....	14
4.5 Consultar valores.....	14
5. Acceso a bases de datos mediante Java.....	15
5.1 Añadir la librería JDBC al proyecto y cargar el driver.....	15
5.2 Clase DriverManager.....	16
5.3 Clase Connection.....	17
5.4 Clase Statement.....	17
5.5 Clase ResultSet.....	19
5.6 Ejemplo completo.....	20
6. Uso de ResultSet: navegabilidad, consultas y modificaciones.....	21
6.1 Navegabilidad y concurrencia.....	21
6.2 Consultas: navegación en un ResultSet.....	22
6.3 Obteniendo datos del ResultSet. Tipos y conversiones.....	23
6.4 Modificación de datos.....	24
6.5 Sentencias y consultas parametrizadas.....	25
6.6 Inserción mediante un ResultSet.....	27
6.7 Borrado con ResultSet.....	28
7. Sentencias SQL con JDBC.....	29
8. Patrones de diseño aplicados al acceso a bases de datos.....	30
8.1 El patrón Singleton.....	30
8.2 El patrón DAO (Data Access Object).....	31
9. Mapeo objeto-relacional.....	34
10. Ejemplo: programa gestor de clientes con XAMPP.....	38
ANEXO I: Conectar a bases de datos Oracle XE.....	41

UD9. ACCESO A BASES DE DATOS

1. INTRODUCCIÓN

 Una **base de datos** es una colección de datos clasificados y estructurados que son guardados en uno o varios ficheros, pero referenciados como si de un único fichero se tratara.

Para crear y manipular bases de datos relacionales, existen en el mercado varios sistemas de gestión de bases de datos (SGBD); por ejemplo: SQL Server, Oracle y DB2. Otros SGBD de libre distribución serían MySQL/MariaDB y PostgreSQL.

Los datos de una base de datos relacional se almacenan en tablas lógicamente relacionadas entre sí utilizando campos clave comunes. A su vez, cada tabla dispone los datos en filas (tuplas ó registros) y columnas (campos).

Los usuarios de un sistema administrador de bases de datos pueden realizar sobre una determinada base operaciones como insertar, recuperar, modificar y eliminar datos, así como añadir nuevas tablas o eliminarlas. Esta operaciones se expresan generalmente en un lenguaje denominado **SQL**.

Antes de empezar **necesitamos un sistema de gestión de base de datos** instalado en nuestro ordenador en el que poder tener bases de datos a las que conectarnos desde nuestros programas escritos en lenguaje Java. Para esta unidad podremos emplear MySQL, Oracle XE ó **XAMPP (Window/Linux Apache MariaDB PHP Perl)** que incorpora un servidor web y la herramienta phpMyAdmin para trabajar con bases de datos. Lo puedes descargar desde la dirección: <https://www.apachefriends.org> Para su instalación tan solo debes seguir los pasos del instalador.

Veréis que las versiones más actuales de XAMPP han sustituido MySQL por MariaDB. No pasa nada, ambos son compatibles y pueden utilizarse indistintamente.

2. REPASO DEL LENGUAJE SQL

En este apartado se da un repaso a los aspectos más relevantes del lenguaje SQL de manipulación de bases de datos relacionales que ya habrás visto en el módulo de Bases de Datos. Te vendrá bien para repasar conceptos. Si no has cursado dicho módulo, es muy importante que leas este apartado con atención.

SQL incluye sentencias tanto de creación de tablas (CREATE) como de manipulación de datos (INSERT, UPDATE y DELETE) así como de consulta de datos (SELECT).

La sintaxis del lenguaje SQL está formado por cuatro grupos sintácticos que pueden combinarse en varias de las sentencias más habituales. Estos 4 grupos son: **Comandos, Cláusulas, Operadores y Funciones**. En este apartado repasaremos solo los tres primeros.

2.1 Comandos

Existen tres tipos de comandos en SQL:

- Los DDL (Data Definition Language), que permiten crear, modificar y borrar nuevas bases de datos, tablas, campos y vistas; trabajando sobre la estructura.
- Los DML (Data Manipulation Language), que permiten introducir información en la BD, borrarla y modificarla, trabajando sobre los datos.
- Los DQL (Data Query Language), que permiten generar consultas para ordenar, filtrar y extraer información de la base de datos (A veces se incluyen dentro del DML).

Los comandos DDL son:

- *CREATE*: Crear nuevas tablas, campos e índices.
- *ALTER*: Modificación de tablas añadiendo campos o modificando la definición de los campos.
- *DROP*: Instrucción para eliminar tablas, campos e índices.

Los comandos DML son:

- *INSERT*: Insertar registros a la base de datos.
- *UPDATE*: Instrucción que modifica los valores de los campos y registros especificados en los criterios.
- *DELETE*: Eliminar registros de una tabla de la base de datos.

El principal comando DQL es:

- *SELECT*: Consulta de registros de la base de datos que cumplen un criterio determinado.

2.2 Cláusulas

Las cláusulas son condiciones de modificación, utilizadas para definir los datos que se desean seleccionar o manipular:

- *FROM*: para especificar la tabla o tablas de las que se seleccionarán los registros.
- *WHERE*: para detallar las condiciones que deben reunir los registros resultantes.
- *GROUP BY*: para separar registros seleccionados en grupos específicos agrupándolos.
- *HAVING*: para expresar la condición que ha de cumplir cada grupo.
- *ORDER BY*: para ordenar los registros seleccionados de acuerdo a un criterio dado.

2.3 Operadores

Operadores lógicos:

- *AND*: devuelve verdadero si ambas condiciones son verdaderas.
- *OR*: devuelve verdadero si alguna de las dos condiciones es cierta.
- *NOT*: Negación lógica. Devuelve el valor contrario a la expresión.

Operadores de comparación:

- *< [...]* menor que [...]
 - *> [...]* mayor que [...]
-

- <> [...] diferente a [...]
- <= [...] menor o igual que [...]
- >= [...] mayor o igual que [...]
- = [...] igual que [...]
- *BETWEEN*: Especifica un intervalo de valores
- *LIKE*: Compara un modelo
- *IN*: Operadores para especificar registros de una tabla

2.4 Ejemplos de sentencias SQL

Para **crear una base de datos**: *CREATE DATABASE <nombre>*

Ejemplo: *CREATE DATABASE tiendaonline*

Para **eliminar una base de datos**: *DROP DATABASE <base de datos>*

Ejemplo: *DROP DATABASE tiendaonline*

Para **utilizar una base de datos**: *USE <base de datos>*

Ejemplo: *USE tiendaonline*

NOTA: Con USE indicamos sobre qué base de datos queremos trabajar (podemos tener varias, consulta con SHOW).

Para **crear una tabla**: *CREATE TABLE <tabla> (<columna 1> [,<columna 2>] ...)*

Donde <columna n> se formula según la siguiente sintaxis:

<columna n> <tipo de dato> [DEFAULT <expresion>] [<const 1> [<const2>]...]

La cláusula *DEFAULT* permite especificar un valor por omisión para la columna y, opcionalmente, para indicar la forma o característica de cada columna, se pueden utilizar las constantes: *NOT NULL*, *UNIQUE* o *PRIMARY KEY*.

La cláusula *PRIMARY KEY* se utiliza para definir la columna como clave principal de la tabla. Esto supone que la columna no puede contener valores nulos ni duplicados. Una tabla puede contener una sola restricción *PRIMARY KEY*.

La cláusula *UNIQUE* indica que la columna no permite valores duplicados. Una tabla puede tener varias restricciones *UNIQUE*.

Ejemplo:

```
CREATE TABLE clientes(
    nombre CHAR(30) NOT NULL,
    direccion CHAR(30) NOT NULL,
    telefono CHAR(12) PRIMARY KEY NOT NULL,
    observaciones CHAR(240)
)
```

Para **borrar una tabla entera**:

DROP TABLE <tabla> Ejemplo: *DROP TABLE clientes*

Para **insertar registros en una tabla**:

INSERT [INTO] <tabla> [(<columna 1> [,<columna 2>] ...)]

```
VALUES (<expresion 1>[,<expresion 2>]...),...
```

Ejemplo: Insertamos dos nuevos registros en la tabla clientes

```
INSERT INTO clientes VALUES ('Pepito Pérez', 'VALENCIA',  
'963003030', 'Ninguna'), ('María Martínez', 'VALENCIA',  
'961002030', 'Ninguna')
```

Para **modificar registros ya existentes en una tabla**:

```
UPDATE <tabla> SET <columna1> = (<expresion1> | NULL) [<columna2  
= ...]...  
WHERE <condición de búsqueda>
```

Ejemplo: Cambiamos el campo 'dirección' del registro cuyo teléfono es '963003030'

```
UPDATE clientes SET direccion = 'Puerto de Sagunto'  
WHERE telefono = '963003030'
```

Para **borrar registros concretos de una tabla (no olvides el WHERE)**:

```
DELETE FROM <tabla> WHERE <condición de búsqueda>
```

Ejemplo: Borramos los registros cuyo campo 'telefono' es 963003030.

```
DELETE FROM clientes WHERE telefono='963003030'
```

Para **realizar una consulta**, es decir, obtener registros de una base de datos:

```
SELECT [ALL | DISTINCT] <lista de selección>  
FROM <tablas>  
WHERE <condiciones de selección>  
[ORDER BY <columna1> [ASC|DESC][, <columna2>[ASC|DESC]]...]
```

Algunos ejemplos sencillos:

Obtenemos todos los registros de la tabla 'clientes':

```
SELECT * FROM clientes;
```

Obtenemos solo el campo 'nombre' de todos los registros de la tabla 'clientes':

```
SELECT nombre FROM clientes;
```

Obtenemos solo los campos 'nombre' y 'telefono' de la tabla 'clientes':

```
SELECT nombre FROM clientes;
```

Obtenemos todos los registros de 'clientes' ordenados por nombre:

```
SELECT * FROM clientes ORDER BY nombre;
```

Obtenemos solo los nombres y teléfonos de los clientes, ordenados por nombre:

```
SELECT nombre, telefono FROM clientes ORDER BY nombre;
```

Obtenemos todos los registros de 'clientes' donde el campo 'telefono' es mayor que 1234

```
SELECT * FROM clientes WHERE telefono > '1234';
```

Obtenemos todos los registros de 'clientes' en los que el campo 'telefono' empieza por 91

```
SELECT * FROM clientes WHERE telefono LIKE '91*';
```

3. JDBC

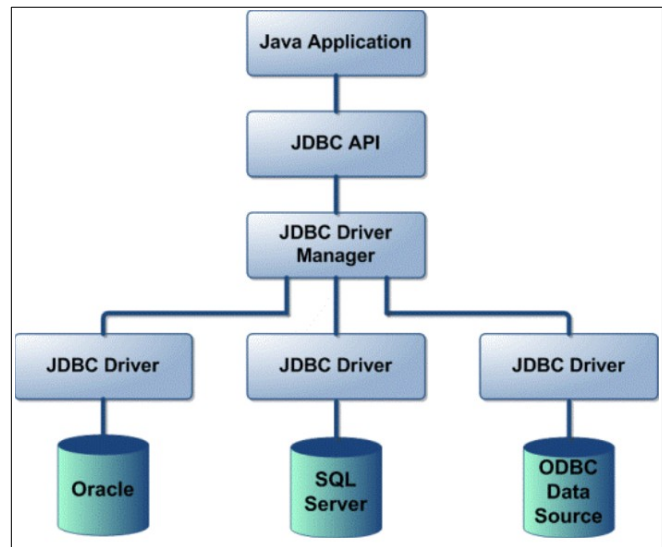
Java puede conectarse con distintos SGBD y también sobre diferentes sistemas operativos. Independientemente del método en que se almacenen los datos debe existir siempre un **mediador** entre la aplicación y el sistema de base de datos y en Java esa función la realiza **JDBC**.



Para la conexión a las bases de datos utilizaremos la API estándar de JAVA denominada **JDBC** (Java Data Base Connection)

JDBC es un API incluido dentro del lenguaje Java para el acceso a bases de datos. Consiste en un conjunto de clases e interfaces escritas en Java que permiten la programación con bases de datos. Se trata de la única solución 100% Java que permite el acceso a bases de datos.

JDBC es una especificación formada por una colección de interfaces y clases abstractas que todos los fabricantes de drivers deben implementar si quieren realizar una implementación de su driver 100% Java y compatible con JDBC (JDBC-compliant driver). Debido a que JDBC está escrito completamente en Java también posee la ventaja de ser independiente de la plataforma.



No será necesario escribir un programa para cada tipo de base de datos, una misma aplicación escrita utilizando JDBC podrá manejar bases de datos Oracle, Sybase, SQL Server, etc.

Además podrá ejecutarse en cualquier sistema operativo que posea una Máquina Virtual de Java, es decir, serán aplicaciones completamente independientes de la plataforma. JDBC tiene sus clases en el paquete *java.sql* y otras extensiones en el paquete *javax.sql*.

Básicamente el API JDBC hace posible la realización de las siguientes tareas:

- Establecer una conexión con una base de datos.
- Enviar sentencias SQL.
- Manipular datos.
- Procesar los resultados de la ejecución de las sentencias.

3.1 Drivers JDBC

Los drivers nos permiten conectarnos con una base de datos determinada. Existen **cuatro tipos de drivers JDBC**, cada tipo presenta una filosofía de trabajo diferente. A continuación describimos cada uno de los drivers:

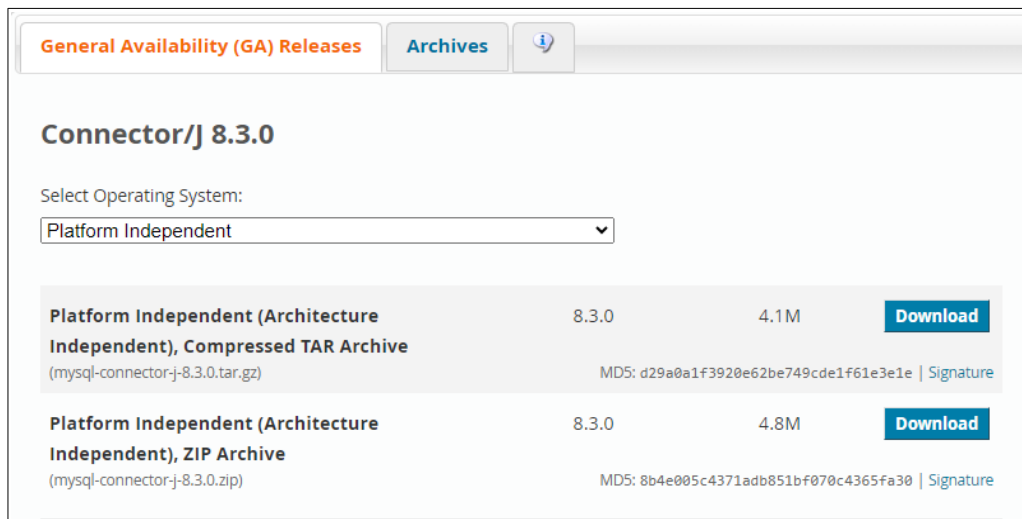
- JDBC-ODBC bridge plus ODBC driver (tipo 1): permite al programador acceder a fuentes de datos ODBC existentes mediante JDBC. El JDBC-ODBC Bridge (puente JDBC-ODBC) implementa operaciones JDBC traduciéndolas a operaciones ODBC. Se encuentra dentro del paquete *sun.jdbc.odbc* y contiene librerías nativas para acceder a ODBC. Al ser usuario de ODBC depende de las dll de ODBC y eso limita la cantidad de plataformas en donde se puede ejecutar la aplicación.
- Native-API partly-Java driver (tipo 2): son similares a los drivers de tipo 1, en tanto y cuanto también necesitan una configuración en la máquina cliente. Este tipo de driver convierte llamadas JDBC a llamadas de Oracle, Sybase, Informix, DB2 u otros SGBD. Tampoco se pueden utilizar dentro de applets al poseer código nativo.
- JDBC-Net pure Java driver (tipo 3): Estos controladores están escritos en Java y se encargan de convertir las llamadas JDBC a un protocolo independiente de la base de datos y en la aplicación servidora utilizan las funciones nativas del sistema de gestión de base de datos mediante el uso de una biblioteca JDBC en el servidor. La ventaja de esta opción es la portabilidad.
- JDBC de Java cliente (tipo 4): Estos controladores están escritos en Java y se encargan de convertir las llamadas JDBC a un protocolo independiente de la base de datos y en la aplicación servidora utilizan las funciones nativas del sistema de gestión de base de datos sin necesidad de bibliotecas. La ventaja de esta opción es la portabilidad. Son como los drivers de tipo 3 pero sin la figura del intermediario y tampoco requieren ninguna configuración en la máquina cliente. Los drivers de tipo 4 se pueden utilizar para servidores Web de tamaño pequeño y medio, así como para intranets. Este tipo es el que vamos a utilizar.

4. ACCESO A BASES DE DATOS MEDIANTE INTERFAZ GRÁFICA (NETBEANS)

Siempre que queramos trabajar con el servidor y las bases de datos del mismo deberemos iniciar la herramienta XAMPP descargada anteriormente e iniciar los servicios Apache y MySQL/MariaDB, o bien tener en marcha un servidor como MySQL 8:

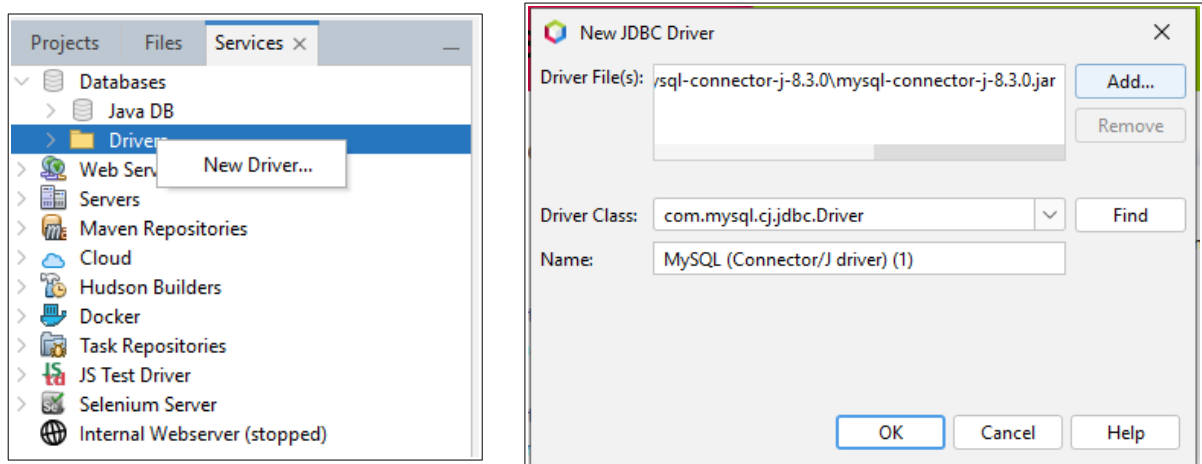
4.1 Instalación del JDBC Driver

Para poder conectarse a una base de datos MySQL/MariaDB desde el explorador de NetBeans necesitamos instalar el driver mysql-connector de Java. Primero hay que descargarlo de la dirección web <http://dev.mysql.com/downloads/connector/j/> En la opción de sistema operativo debemos seleccionar **"Platform Independent"** y luego descargar el ZIP.



No hace falta registrarse para descargarlo, solo hay pulsar en *“No thanks, just start my download.”* Descomprime el ZIP y busca el archivo **mysql-connector-java-8.3.0.jar**. Ten en cuenta que el número de versión puede ser diferente a la que se muestra en la imagen.

Ahora **abre NetBeans**, ve al **panel Services → Databases**, haz **clic derecho en la opción Drivers** y selecciona la opción **New Driver** del menú contextual. Se mostrará el diálogo *New JDBC Driver*.



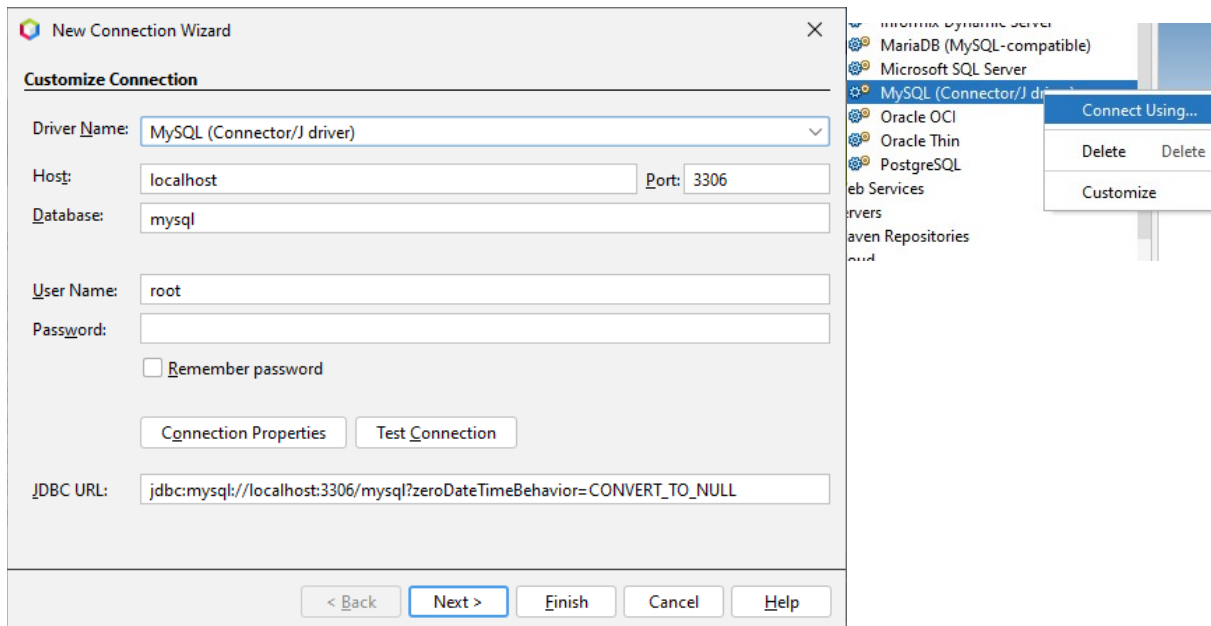
Hac **clic en Add...** y selecciona el fichero **mysql-connector-java-8.3.0.jar** que has descargado anteriormente. Una vez seleccionado el fichero nos debe de rellenar la clase principal del Driver que en nuestro caso debe de ser **com.mysql.cj.jdbc.Driver** y el nombre a nuestro driver, por ejemplo **MySQL (Connector/J driver)**.

NOTA: Si estás utilizando la última versión de NetBeans es posible que en Services → Databases → Drivers ya te aparezca la opción ‘MySQL (Connector/J driver)’. A pesar de ello puede que necesites instalar el driver. Dale al driver con clic derecho → ‘Connect using’ y comprueba si en la ventana te aparece el mensaje ‘driver file is missing’. En tal caso, dale a ‘add driver’ y selecciona el archivo jar mencionado anteriormente.

Llegados a este punto ya tenemos enlazado el driver JDBC de MySQL/MariaDB desde el Database Explorer. Ahora lo que nos queda es crear una conexión con una base de datos MySQL/MariaDB.

4.2 Conexión a Base de Datos mediante el JDBC Driver

Haz clic derecho en 'MySQL (Connector/J driver)' y selecciona la opción 'Connect Using...'.



Aparecerá la ventana 'New Connection Wizard':

- **Driver Name:** El driver que queremos utilizar. En este caso 'MySQL (Connector/J driver)'.
- **Host:** La dirección IP o nombre de dominio de la máquina donde se encuentra instalado el servidor MySQL. En nuestro caso 'localhost' (máquina local).
- **Port:** Puerto de conexión. MySQL utiliza el 3306 por defecto.
- **Database:** El nombre de la base de datos a la que queremos acceder. Podemos dejar este campo vacío para poder acceder a todas las que existan en el servidor.
- **User Name:** Nombre de usuario. Utilizaremos 'root' (administrador) o el que corresponda.
- **Password:** Contraseña del usuario.
- **JDBC URL:** La URL de conexión con el formato `jdbc:mysql://HOST:PUERTO`. Se crea automáticamente a partir de la información anterior. Es posible que incluya por defecto variables como 'zeroDateTimeBehaviour' en la imagen.

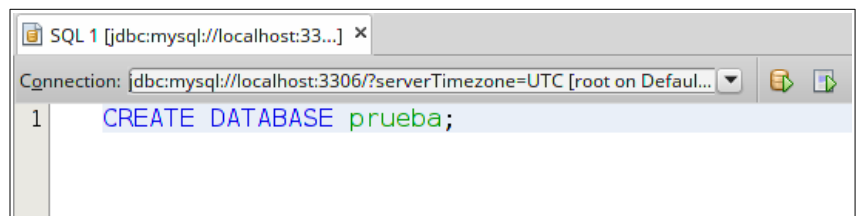
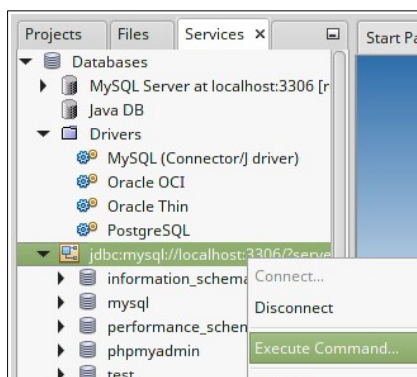
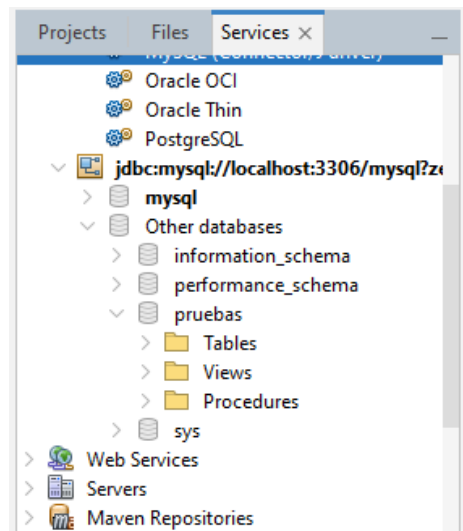
Haz clic en **Finalizar**. Si todo va bien, en Services → Databases aparecerá una nueva conexión llamada `jdbc:mysql://localhost:3306... [root on Default schema]`

NOTA: Si falla debido a un error 'server time zone' puedes solucionarlo añadiendo al URL la variable `serverTimezone` con el valor UTC: `jdbc:mysql://localhost:3306/?serverTimezone=UTC`

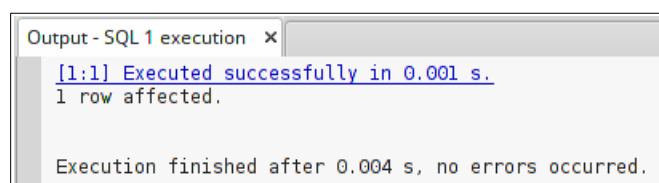
Si desplegamos la conexión creada veremos que podemos acceder a todas las bases de datos, tablas, etc. almacenadas en el servidor al que estamos conectados (a cada uno le aparecerán las bases de datos que tenga en su sistema, pueden ser diferentes a las de la imagen).

Esto es muy útil ya que nos permitirá acceder cómodamente a la base de datos, realizar consultas, ejecutar comandos, etc. mientras desarrollamos un proyecto con NetBeans.

Para ejecutar un comando sobre la base de datos a la que nos hemos conectado solo hay que hacer clic derecho sobre la conexión y seleccionar la opción 'Execute Command...'. Se abrirá un panel en el editor central en el que podremos introducir sentencias SQL (tantas como queramos) y ejecutarlas haciendo click en el icono 'Run SQL':



Prueba a ejecutar la sentencia **CREATE DATABASE prueba;** como en la imagen. En el panel de abajo se mostrará un mensaje con el resultado de la ejecución. En este caso indica 'Executed successfully' (se ha ejecutado satisfactoriamente). En la conexión jdbc del panel de la izquierda aparecerá la nueva base de datos 'prueba' que hemos creado.



Hay que tener en cuenta que como hemos configurado esta conexión sin especificar ninguna base de datos concreta, si quisiéramos ejecutar sentencias sobre una base de datos de las disponibles tendríamos que utilizar siempre la sentencia `USE nombre_de_base_de_datos`. Por ejemplo:

```
USE tiendaonline;  
SELECT * FROM clientes;
```

Si vamos a trabajar habitualmente sobre la misma base de datos y queremos evitar tener que escribir siempre la sentencia `USE` en todas las ejecuciones de sentencias SQL, podemos crear una conexión configurada para utilizar una base de datos concreta. Para ello primero tendríamos que

seguir el mismo procedimiento explicado en este apartado (Clic derecho en 'MySQL (Connector/J driver)' → 'Connect Using...' → Introducir los datos en el diálogo 'New Connection Wizard' y en el campo 'Database' escribir la base de datos deseada.

Por ejemplo, para el caso de la base de datos prueba que hemos creado, la conexión generaría una URL resultante tal que así: **`jdbc:mysql://localhost:3306/prueba?serverTimezone=UTC`**

En los siguientes apartados veremos algunos ejemplos de ejecución de sentencias SQL utilizando esta nueva conexión a la base de datos 'prueba'.

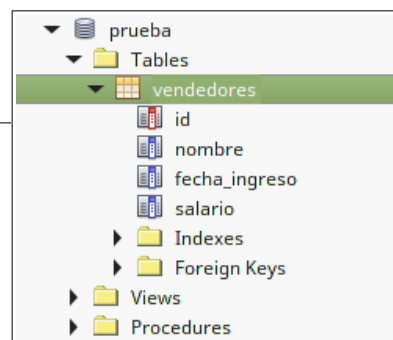
4.3 Crear una tabla

Desde el IDE de NetBeans podemos crear tablas de dos formas: por sentencias SQL o utilizando un asistente de creación de tablas.

4.3.1 Mediante sentencias SQL

Seleccionamos la conexión `jdbc:mysql://localhost:3306/prueba` creada en el apartado anterior. Hacemos clic derecho sobre ella → *Execute Command...* e introducimos el siguiente comando SQL:

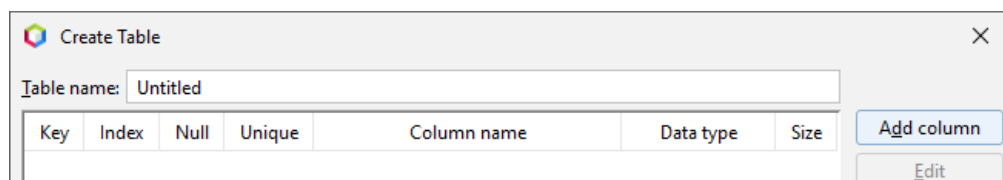
```
CREATE TABLE vendedores (  
    id int NOT NULL auto_increment,  
    nombre varchar(50) NOT NULL default '',  
    fecha_ingreso date NOT NULL default '0000-00-00',  
    salario float NOT NULL default '0',  
    PRIMARY KEY (id)  
);
```



Si todo ha ido bien dentro de 'prueba → Tables' veremos la nueva tabla. Es posible que necesitemos hacer clic derecho → *Refresh* (sobre prueba) para que se actualice la información.

4.3.2 Mediante el asistente de creación de tablas

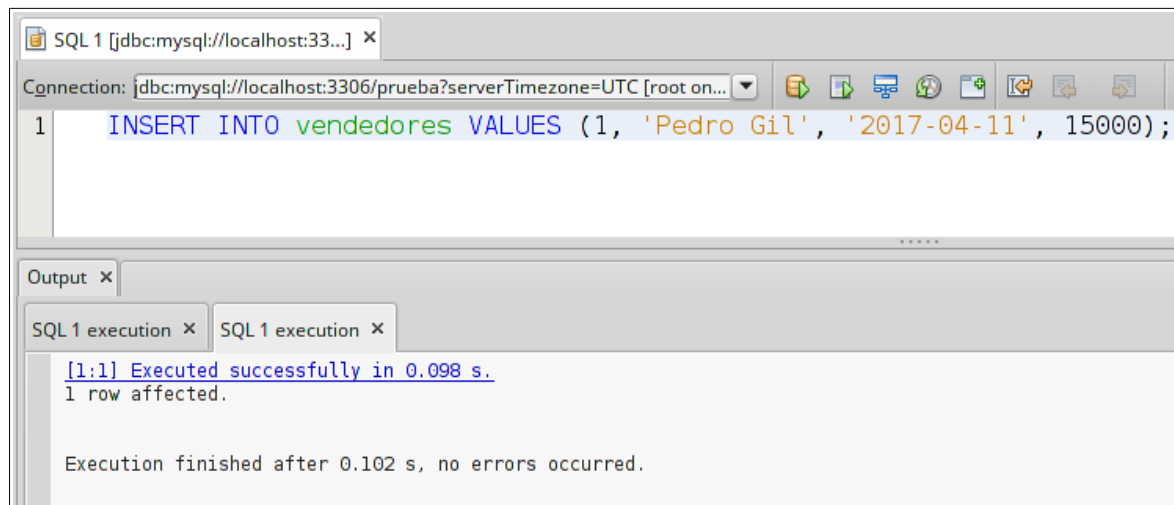
Sobre la carpeta 'Tables' de la conexión deseada, hacemos clic derecho y elegimos '*Create Table*'. Se abrirá el asistente de creación de tablas. Añadimos todas las columnas que necesitemos mediante la opción 'Add Column' (indicando las opciones de cada columna) y hacemos clic en 'OK'.



4.4 Insertar datos

Ejecutar el comando INSERT INTO... sobre la conexión deseada. Por ejemplo:

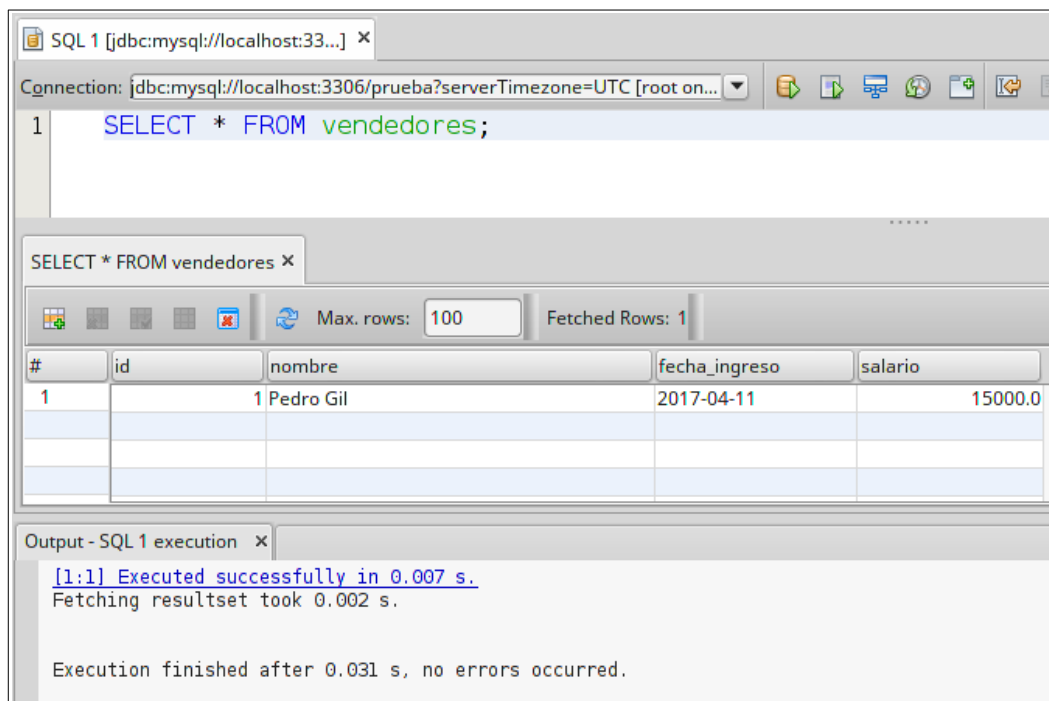
```
INSERT INTO vendedores VALUES (1, 'Pedro Gil', '2017-04-11', 15000);
```



4.5 Consultar valores

Ejecutar el comando SELECT... sobre la conexión deseada. Por ejemplo:

```
SELECT * FROM vendedores;
```

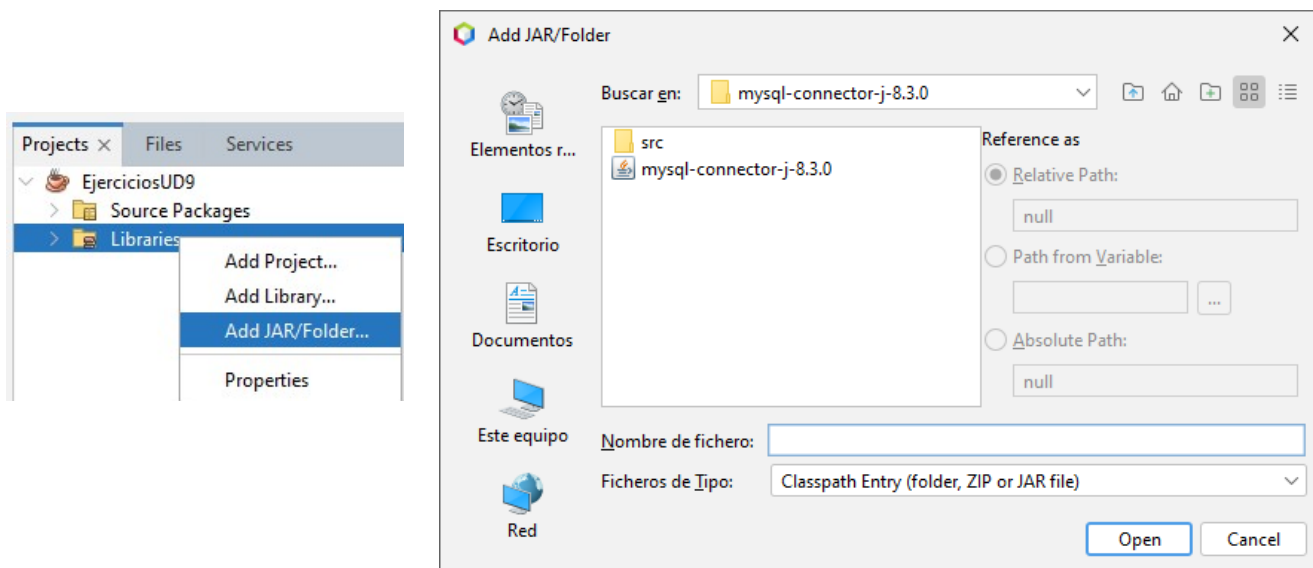


5. ACCESO A BASES DE DATOS MEDIANTE JAVA

En este apartado se ofrece una introducción a los aspectos fundamentales del acceso a bases de datos mediante código Java. En los siguientes apartados se explicarán algunos aspectos en mayor detalle, sobre todo los relacionados con las clases Statement y ResultSet.

5.1 Añadir la librería JDBC al proyecto y cargar el driver

Para poder utilizar la librería JDBC en Java primero deberemos añadirla al proyecto. Para ello debemos hacer clic derecho sobre la carpeta 'Libraries' del proyecto y seleccionar 'Add JAR/Folder'. En la ventana emergente deberemos seleccionar el archivo del driver previamente descargado mysql-connector-java-8.3.0.jar y clic en OK.



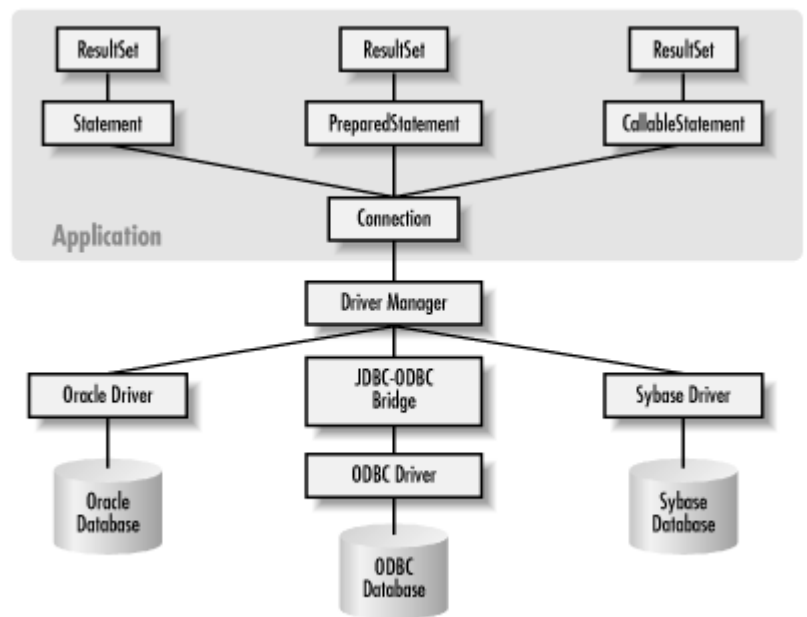
En un proyecto Java que realice conexiones a bases de datos es necesario, antes que nada, utilizar *Class.forName(...)* para cargar dinámicamente el driver que vamos a utilizar. Esto solo es necesario hacerlo una vez en nuestro programa. Puede lanzar excepciones controladas por lo que es obligatorio utilizar un bloque try-catch. Veamos un ejemplo con el driver de MySQL:

```
try {
    Class.forName("com.mysql.cj.jdbc.Driver");
} catch (Exception e) {
    // manejamos el error
}
```

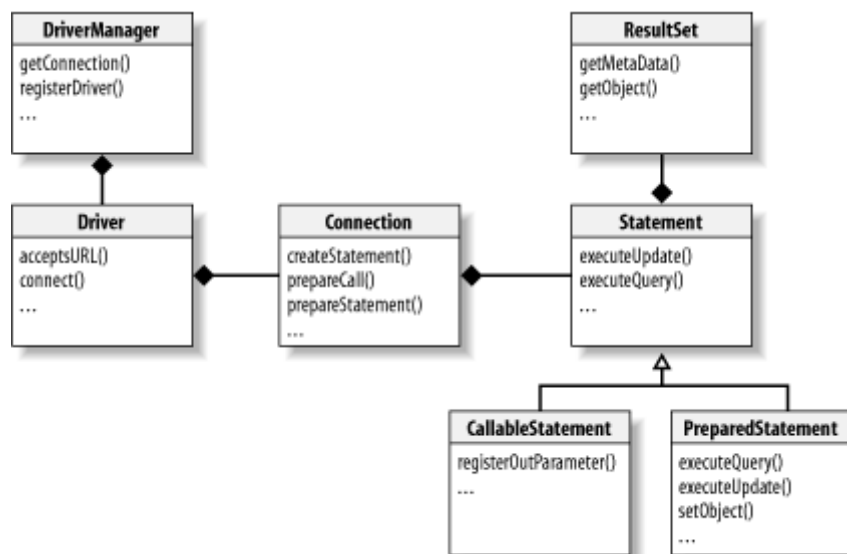
ATENCIÓN: Desde la versión 6 de Java y 4 del JDBC, ya **no es necesario** "cargar" el driver explícitamente, sin embargo, es importante conocer su existencia y saber, ante una necesidad, cómo se haría.

Hay que tener en cuenta que las clases y métodos utilizados para conectarse a una base de datos (explicados más adelante) funcionan con todos los drivers disponibles para Java. Esto es posible ya que el estándar de Java solo los define como interfaces (*interface*) y cada librería "driver" los implementa (define las clases y su código). Por ello es necesario utilizar *Class.forName(...)* para indicarle a Java qué driver vamos a utilizar.

Este nivel de abstracción facilita el desarrollo de proyectos ya que si necesitáramos utilizar otro sistema de base de datos (que no fuera MySQL) solo necesitaríamos cambiar la línea de código que carga el driver y poco más. Si cada sistema de base de datos necesitara que utilizáramos distintas clases y métodos todo sería mucho más complicado.



Las cuatro clases fundamentales que toda aplicación Java necesita para conectarse a una base de datos y ejecutar sentencias son: **DriverManager**, **Connection**, **Statement** y **ResultSet**.



5.2 Clase DriverManager

La clase **java.sql.DriverManager** es la capa gestora del driver JDBC. Se encarga de manejar el Driver apropiado y **permite crear conexiones con una base de datos** mediante el método estático **getConnection(...)** que tiene dos variantes:

```
DriverManager.getConnection(String url);
DriverManager.getConnection(String url, String user, String password);
```


Este método intentará establecer una conexión con la base de datos según la URL indicada. Opcionalmente se le puede pasar el usuario y contraseña como argumento (también se puede indicar en la propia URL). Si la conexión es satisfactoria devolverá un objeto **Connection**.

Ejemplo de conexión a la base de datos 'prueba' en localhost:

```
String url = "jdbc:mysql://localhost:3306/prueba";  
Connection conn = DriverManager.getConnection(url,"root","123456");
```

Este método puede lanzar dos tipos de excepciones (que habrá que manejar con un try-catch):

- **SQLException**: La conexión no ha podido producirse. Puede ser por multitud de motivos como una URL mal formada, un error en la red, host o puerto incorrecto, base de datos no existente, usuario y contraseña no válidos, etc.
- **SQLTimeoutException**: Se ha superado el LoginTimeout sin recibir respuesta del servidor.

5.3 Clase Connection

Un objeto **java.sql.Connection** representa una sesión de conexión con una base de datos. Una aplicación puede tener tantas conexiones como necesite, ya sea con una o varias bases de datos; aunque generalmente se genera únicamente una conexión mediante el patrón **Singleton**.

El método más relevante es **createStatement()** que devuelve un objeto Statement asociado a dicha conexión que permite ejecutar sentencias SQL. El método **createStatement()** puede lanzar excepciones de tipo **SQLException**.

```
Statement stmt = conn.createStatement();
```

Cuando ya no la necesitemos es aconsejable **cerrar la conexión con close()** para liberar recursos.

```
conn.close();
```

5.4 Clase Statement

Un objeto **java.sql.Statement** permite **ejecutar sentencias SQL en la base de datos** a través de la conexión con la que se creó el Statement (ver apartado anterior). Los tres métodos más comunes de ejecución de sentencias SQL son **executeQuery(...)**, **executeUpdate(...)** y **execute(...)**. Todas ellas pueden lanzar excepciones de tipo **SQLException** y **SQLTimeoutException**.

- **ResultSet executeQuery(String sql)**: Ejecuta la sentencia sql indicada (de tipo SELECT). Devuelve un objeto ResultSet con los datos proporcionados por el servidor.

```
ResultSet rs = stmt.executeQuery("SELECT * FROM vendedores");
```

- **int executeUpdate(String sql)**: Ejecuta la sentencia sql indicada (de tipo DML como por ejemplo INSERT, UPDATE o DELETE). Devuelve un el número de registros que han sido

insertados, modificados o eliminados.

```
String sql = INSERT INTO clientes VALUES (1,'Pedro Gil');  
int nr = stmt.executeUpdate(sql);
```

- **boolean execute(String sql):** este método puede ejecutar cualquier tipo de consulta SQL: ya sea una consulta de selección (SELECT), una consulta de modificación de datos (INSERT, UPDATE, DELETE), o una instrucción DDL (Data Definition Language) como CREATE TABLE, DROP TABLE, etc. Sin embargo, suele usarse más con instrucciones que no devuelvan ResultSet, como las de tipo DDL.

```
String sql = "CREATE TABLE ejemplo (id INT, nombre VARCHAR(50))";  
boolean resultado = statement.execute(sql);
```

Cuando ya no lo necesitemos es aconsejable **cerrar el statement con close()** para liberar recursos.

```
stmt.close();
```

5.5 Clase ResultSet

Un objeto `java.sql.ResultSet` contiene un conjunto de resultados (datos) obtenidos tras ejecutar una sentencia SQL, normalmente de tipo SELECT. Es una **estructura de datos en forma de tabla con registros (filas)** que podemos recorrer para acceder a la información de sus **campos (columnas)**.

ResultSet utiliza internamente un cursor que apunta al 'registro actual' sobre el que podemos operar. Inicialmente dicho cursor está situado **antes** de la primera fila y disponemos de varios métodos para desplazar el cursor. El más común es `next()`:

- **boolean next()**: Mueve el cursor al siguiente registro. Devuelve true si fué posible y false en caso contrario (si ya llegamos al final de la tabla).

Algunos de los métodos para **obtener los datos del registro actual** son:

- **String getString(String columnLabel)**: Devuelve un dato String de la columna indicada por su nombre. Por ejemplo: `rs.getString("nombre")`
- **String getString(int columnIndex)**: Devuelve un dato String de la columna indicada por su nombre. La primera columna es la 1, no la cero. Por ejemplo: `rs.getString(2)`

Existen métodos análogos a los anteriores para obtener valores de tipo int, long, float, double, boolean, Date, Time, Array, etc. Pueden consultarse todos en la [documentación oficial de Java](#).

- **int getInt(String columnLabel)** **int getInt(int columnIndex)**
- **double getDouble(String columnLabel)** **double getDouble(int columnIndex)**
- **boolean getBoolean(String columnLabel)** **boolean getBoolean(int columnIndex)**
- **Date getDate(String columnLabel)** **int getDate(int columnIndex)**
- etc...

Más adelante veremos cómo se realiza la modificación e inserción de datos. Todos estos métodos pueden lanzar una **SQLException**.

Veamos un ejemplo de cómo recorrer un ResultSet llamado `rs` y mostrarlo por pantalla:

```
while(rs.next()) {  
    int id      = rs.getInt("id");  
    String nombre = rs.getString("nombre");  
    Date fecha   = rs.getDate("fecha_ingreso");  
    float salario = rs.getFloat("salario");  
    System.out.println(id + " " + nombre + " " + fecha + " " + salario);  
}
```

5.6 Ejemplo completo

Veamos un ejemplo completo de conexión y acceso a una base de datos utilizando todos los elementos mencionados en este apartado, comentado para poder entenderlo mejor:

```
try {
    // Cargamos el driver
    Class.forName("com.mysql.cj.jdbc.Driver");

    // Creamos una nueva conexión a la base de datos 'prueba'
    String url = "jdbc:mysql://localhost:3306/prueba?serverTimezone=UTC";
    Connection conn = DriverManager.getConnection(url,"root","123456");

    // Obtenemos un Statement de la conexión
    Statement st = conn.createStatement();

    // Ejecutamos una consulta SELECT para obtener la tabla vendedores
    String sql = "SELECT * FROM vendedores";
    ResultSet rs = st.executeQuery(sql);

    // Recorremos todo el ResultSet y mostramos sus datos
    while(rs.next()) {
        int id = rs.getInt("id");
        String nombre = rs.getString("nombre");
        Date fecha = rs.getDate("fecha_ingreso");
        float salario = rs.getFloat("salario");
        System.out.println(id + " " + nombre + " " + fecha + " " + salario);
    }

    // Cerramos el statement y la conexión
    st.close();
    conn.close();
} catch (SQLException e) {
    e.printStackTrace();
} catch (Exception e) {
    e.printStackTrace();
}
```

6. USO DE RESULTSET: NAVEGABILIDAD, CONSULTAS Y MODIFICACIONES

6.1 Navegabilidad y concurrencia

Cuando invocamos a **`createStatement()`** sin argumentos, como hemos visto anteriormente, al ejecutar sentencias SQL obtendremos un **ResultSet por defecto en el que el cursor solo puede moverse hacia adelante y los datos son de solo lectura**. A veces esto no es suficiente y necesitamos mayor funcionalidad. Por ello el método **`createStatement()`** está sobrecargado (existen varias versiones de dicho método) lo cual nos permite invocarlo con argumentos en los que podemos especificar el funcionamiento.

- **`Statement createStatement(int resultSetType, int resultSetConcurrency)`**: devuelve un objeto Statement cuyos objetos ResultSet serán del tipo y concurrencia especificados. Los valores válidos son constantes definidas en ResultSet.

El argumento **`resultSetType`** indica el tipo de ResultSet:

- **`ResultSet.TYPE_FORWARD_ONLY`**: ResultSet por defecto, forward-only y no-actualizable.
 - Solo permite movimiento hacia delante con `next()`.
 - Sus datos NO se actualizan. Es decir, no reflejará cambios producidos en la base de datos. Contiene una instantánea del momento en el que se realizó la consulta.
- **`ResultSet.TYPE_SCROLL_INSENSITIVE`**: ResultSet desplazable y no actualizable.
 - Permite libertad de movimiento del cursor con otros métodos como `first()`, `previous()`, `last()`, etc. además de `next()`.
 - Sus datos NO se actualizan, como en el caso anterior.
- **`ResultSet.TYPE_SCROLL_SENSITIVE`**: ResultSet desplazable y actualizable.
 - Permite libertad de movimientos del cursor, como en el caso anterior.
 - Sus datos SÍ se actualizan. Es decir, mientras el ResultSet esté abierto se actualizará automáticamente con los cambios producidos en la base de datos. Esto puede suceder incluso mientras se está recorriendo el ResultSet, lo cual puede ser conveniente o contraproducente según el caso.

El argumento **`resultSetConcurrency`** indica la concurrencia del ResultSet:

- **`ResultSet.CONCUR_READ_ONLY`**: Solo lectura. Es el valor por defecto.
- **`ResultSet.CONCUR_UPDATABLE`**: Permite modificar los datos almacenados en el ResultSet para luego aplicar los cambios sobre la base de datos (más adelante se verá cómo).

El **ResultSet** por defecto que se obtiene con **`createStatement()`** sin argumentos es el mismo que con **`createStatement(ResultSet.TYPE_FORWARD_ONLY, ResultSet.CONCUR_READ_ONLY)`**.

6.2 Consultas: navegación en un ResultSet

Como ya se ha visto, en un objeto *ResultSet* se encuentran los resultados de la ejecución de una sentencia SQL. Por lo tanto, un objeto *ResultSet* contiene las filas que satisfacen las condiciones de una sentencia SQL, y ofrece métodos de navegación por los registros como `next()` que desplaza el curso al siguiente registro del *ResultSet*.

Además de este método de desplazamiento básico, existen otros de desplazamiento libre que podremos utilizar siempre y cuando el *ResultSet* sea de tipo *ResultSet.TYPE_SCROLL_INSENSITIVE* o *ResultSet.TYPE_SCROLL_SENSITIVE* como se ha dicho antes.

Algunos de estos métodos son:

- **void beforeFirst():** Mueve el cursor antes de la primera fila.
- **boolean first():** Mueve el cursor a la primera fila.
- **boolean next():** Mueve el cursor a la siguiente fila. Permitido en todos los tipos de *ResultSet*.
- **boolean previous():** Mueve el cursor a la fila anterior.
- **boolean last():** Mueve el cursor a la última fila.
- **void afterLast():** Mover el cursor después de la última fila.
- **boolean absolute(int row):** Posiciona el cursor en el número de registro indicado. Hay que tener en cuenta que el primer registro es el 1, no el cero. Por ejemplo `absolute(7)` desplazará el cursor al séptimo registro. Si valor es negativo se posiciona en el número de registro indicado pero empezando a contar desde el final (el último es el -1). Por ejemplo si tiene 10 registros y llamamos `absolute(-2)` se desplazará al registro n.º 9.
- **boolean relative(int registros):** Desplaza el cursor un número relativo de registros, que puede ser positivo o negativo. Por ejemplo si el cursor está en el registro 5 y llamamos a `relative(10)` se desplazará al registro 15. Si luego llamamos a `relative(-4)` se desplazará al registro 11.

Los métodos que devuelven un tipo boolean devolverán 'true' si ha sido posible mover el cursor a un registro válido, y 'false' en caso contrario, por ejemplo si no tiene ningún registro o hemos saltado a un número de registro que no existe.

Todos estos métodos pueden producir una excepción de tipo **SQLException**.

También existen otros métodos relacionados con la posición del cursor.

- **int getRow():** Devuelve el número de registro actual. Cero si no hay registro actual.
 - **boolean isBeforeFirst():** Devuelve 'true' si el cursor está antes del primer registro.
 - **boolean isFirst():** Devuelve 'true' si el cursor está en el primer registro.
 - **boolean isLast():** Devuelve 'true' si el cursor está en el último registro.
 - **boolean isAfterLast():** Devuelve 'true' si el cursor está después del último registro.
-

6.3 Obteniendo datos del ResultSet. Tipos y conversiones

Los métodos *getXXX()* ofrecen los medios para recuperar los valores de las columnas (campos) de la fila (registro) actual del *ResultSet*. No es necesario que las columnas sean obtenidas utilizando un orden determinado.

Para designar una columna podemos utilizar su nombre o bien su número (empezando por 1).

Por ejemplo si la segunda columna de un objeto *ResultSet* se llama "título" y almacena datos de tipo *String*, se podrá recuperar su valor de las dos formas siguientes:

```
// rs es un objeto ResultSet
String valor = rs.getString(2);
String valor = rs.getString("titulo");
```

Es importante tener en cuenta que las columnas se numeran de izquierda a derecha y que la primera es la número 1, no la cero. También que las columnas no son case sensitive, es decir, no distinguen entre mayúsculas y minúsculas.



La información referente a las columnas de un *ResultSet* se puede obtener llamando al **método *getMetaData()*** que devolverá un objeto *ResultSetMetaData* que contendrá el número, tipo y propiedades de las columnas del *ResultSet*.

Si conocemos el nombre de una columna, pero no su índice, el método ***findColumn()*** puede ser utilizado para obtener el número de columna, pasándole como argumento un objeto *String* que sea el nombre de la columna correspondiente, este método nos devolverá un entero que será el índice correspondiente a la columna.

Cuando se lanza un método *getXXX()* determinado sobre un objeto *ResultSet* para obtener el valor de un campo del registro actual, el driver JDBC convierte el dato que se quiere recuperar al tipo Java especificado y entonces devuelve un valor Java adecuado. Por ejemplo si utilizamos el método *getString()* y el tipo del dato en la base de datos es *VARCHAR*, el driver JDBC convertirá el dato *VARCHAR* de tipo SQL a un objeto *String* de Java.

Algo parecido sucede con otros tipos de datos SQL como por ejemplo *DATE*. Podremos acceder a él tanto con *getDate()* como con *getString()*. La diferencia es que el primero devolverá un objeto Java de tipo *Date* y el segundo devolverá un *String*.

Siempre que sea posible el driver JDBC convertirá el tipo de dato almacenado en la base de datos al tipo solicitado por el método *getXXX()*, pero hay conversiones que no se pueden realizar y lanzarán una excepción, como por ejemplo si intentamos hacer un *getInt()* sobre un campo que no contiene un valor numérico.

6.4 Modificación de datos

Para poder modificar los datos que contiene un *ResultSet* necesitamos un *ResultSet* de tipo modificable. Para ello debemos utilizar la constante *ResultSet.CONCUR_UPDATABLE* al llamar al método *createStatement()* como se ha visto antes.

Para modificar los valores de un registro existente se utilizan una serie de métodos *updateXXX()* de *ResultSet*. Las *XXX* indican el tipo del dato y hay tantos distintos como sucede con los métodos *getXXX()* de este mismo interfaz: ***updateString()*, *updateInt()*, *updateDouble()*, *updateDate()*, etc.**

La diferencia es que **los métodos *updateXXX()* necesitan dos argumentos:**

- **La columna que deseamos actualizar** (por su nombre o por su numero de columna).
- **El valor que queremos almacenar** en dicha columna (del tipo que sea).

Por ejemplo para modificar el campo 'edad' almacenando el entero 28 habría que llamar al siguiente método, suponiendo que *rs* es un objeto *ResultSet*:

```
rs.updateInt("edad", 28);
```

También podría hacerse de la siguiente manera, suponiendo que la columna "edad" es la segunda:

```
rs.updateInt(2, 28);
```

Los métodos *updateXXX()* no devuelven ningún valor (son de tipo *void*). Si se produce algún error se lanzará una *SQLException*.

Posteriormente hay que **llamar a *updateRow()* para que los cambios realizados se apliquen sobre la base de datos**. El Driver JDBC se encargará de ejecutar las sentencias SQL necesarias. Esta es una característica muy potente ya que nos facilita enormemente la tarea de modificar los datos de una base de datos. Este método devuelve *void*.

En resumen, el proceso para realizar la modificación de una fila de un *ResultSet* es el siguiente:

1. **Desplazamos el cursor al registro** que queremos modificar.
2. Llamamos a todos los métodos ***updateXXX(...)*** que necesitamos.
3. Llamamos a ***updateRow()*** para que los cambios se apliquen a la base de datos.

Es importante entender que **hay que llamar a *updateRow()* antes de desplazar el cursor**. Si desplazamos el cursor antes de llamar a *updateRow()*, se perderán los cambios.

Si queremos **cancelar las modificaciones de un registro del *ResultSet*** podemos llamar a ***cancelRowUpdates()***, que cancela todas las modificaciones realizadas sobre el registro actual.

Si ya hemos llamado a *updateRow()* el método *cancelRowUpdates()* no tendrá ningún efecto.

El siguiente código de ejemplo muestra cómo modificar el campo 'dirección' del último registro de un ResultSet que contiene el resultado de una SELECT sobre la tabla de clientes. Supondremos que `conn` es un objeto Connection previamente creado:

```
// Creamos un Statement scrollable y modificable
Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                       ResultSet.CONCUR_UPDATABLE);

// Ejecutamos un SELECT y obtenemos la tabla clientes en un ResultSet
String sql = "SELECT * FROM clientes";
ResultSet rs = stmt.executeQuery(sql);

// Vamos al último registro, lo modificamos y actualizamos la base de datos
rs.last();
rs.updateString("direccion", "C/ Pepe Ciges, 3");
rs.updateRow();
```

Las modificaciones también pueden realizarse lanzando directamente sentencias UPDATE, veamos un ejemplo:

```
String sql = "UPDATE productos SET precio = ? WHERE nombre = ?";
PreparedStatement sentencia = null;

try {
    sentencia = conexion.prepareStatement(sql);
    sentencia.setFloat(1, precioProducto);
    sentencia.setString(2, nombreProducto);
    sentencia.executeUpdate();
} catch (SQLException sqle) {
    sqle.printStackTrace();
} finally {
    if (sentencia != null)
        try {
            sentencia.close();
        } catch (SQLException sqle) {
            sqle.printStackTrace();
        }
}
```

Se requiere de la existencia de 1 objeto de tipo Connection con nombre de variable "conexión" y 2 variables donde se han recibido los datos a introducir en el UPDATE. Como vemos, aparece un objeto de JDBC nuevo: PreparedStatement. Esto nos va a permitir emplear sentencias parametrizadas:

6.5 Sentencias y consultas parametrizadas

Las sentencias parametrizadas son cláusulas SQL donde podemos introducir modificadores, generalmente variables, que provienen de algún sitio (como una lectura de teclado, o una introducción desde un jTextField). Esto sirve para que el usuario pueda interactuar indicando qué valores y condiciones deben cumplirse en inserciones, modificaciones y borrados; aunque también

se puede emplear en consultas, llamándolas entonces consultas parametrizadas.

Además, el uso de consultas parametrizadas elimina el problema de SQL-Injection, que es un fallo de seguridad importante en el acceso a bases de datos.

Estas consultas no concatenan las variables a la consulta SQL sino que usan una sintaxis específica para pasar a la consulta SQL un conjunto de parámetros predeterminado. Vamos a ver a continuación un ejemplo:

```
Connection conexion = null;
PreparedStatement sentencia = null;
ResultSet rs = null;
boolean valido = false;

Class.forName("com.mysql.jdbc.Driver");
String url = "jdbc:mysql://localhost:3306/bd1";
conexion = DriverManager.getConnection(url,"root","password");

String sql = "select nombre from users where user =? and pass =?";
sentencia = conexion.prepareStatement(sql);
sentencia.setString(1, user);
sentencia.setString(2, pass);
rs = sentencia.executeQuery();

if (rs.next())
    valido = true;
```

Como podemos ver, la consulta se construye utilizando el carácter de interrogación “?” para definir la posición de cada uno de los parámetros. El valor de los parámetros será pasado mas adelante a través de los métodos “setString()” y otros similares. Este aspecto es aplicable tanto en UPDATE, DELETE, INSERT como con SELECT; y en general cualquier sentencia SQL proclive al uso de parámetros mapeables con variables u otras fuentes de datos.

6.6 Inserción mediante un ResultSet

Para insertar nuevos registros necesitaremos utilizar, al menos, estos dos métodos:

- **void moveToInsertRow():** Desplaza el cursor al 'registro de inserción'. Es un registro especial utilizado para insertar nuevos registros en el ResultSet. Posteriormente tendremos que llamar a los métodos updateXXX() ya conocidos para establecer los valores del registro de inserción. Para finalizar hay que llamar a insertRow().
- **void insertRow():** Inserta el 'registro de inserción' en el ResultSet, pasando a ser un registro normal más, y también lo inserta en la base de datos.

El siguiente código inserta un nuevo registro en la tabla 'clientes'. Supondremos que `conn` es un objeto Connection previamente creado:

```
// Creamos un Statement scrollable y modificable
Statement stmt =
conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                      ResultSet.CONCUR_UPDATABLE);

// Ejecutamos un SELECT y obtenemos la tabla clientes en un ResultSet
String sql = "SELECT * FROM clientes";
ResultSet rs = stmt.executeQuery(sql);

// Creamos un nuevo registro y lo insertamos
rs.moveToInsertRow();
rs.updateString(2,"Killy Lopez");
rs.updateString(3,"Wall Street 3674");
rs.insertRow();
```

Los campos cuyo valor no se haya establecido con updateXXX() tendrán un valor NULL. Si en la base de datos dicho campo no está configurado para admitir nulos se producirá una SQLException.

Tras insertar nuestro nuevo registro en el objeto ResultSet podremos volver a la anterior posición en la que se encontraba el cursor (antes de invocar moveToInsertRow()) llamando al método moveToCurrentRow(). Este método sólo se puede utilizar en combinación con moveToInsertRow().

Como ves, este apartado no usa las sentencias parametrizadas con INSERT, sino que usa updateString sobre el ResultSet y luego lo inserta, investiga cómo lanzarías directamente un INSERT al JDBC. Reflexiona, ¿qué crees que es más conveniente, la opción del ejemplo o lanzar directamente el INSERT?, ¿porqué?

6.7 Borrado con ResultSet

Para eliminar un registro solo hay que desplazar el cursor al registro deseado y llamar al método:

- **void deleteRow()**: Elimina el registro actual del ResultSet y también de la base de datos.

El siguiente código borra el tercer registro de la tabla 'clientes':

```
// Creamos un Statement scrollable y modificable
Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,
                                       ResultSet.CONCUR_UPDATABLE);

// Ejecutamos un SELECT y obtenemos la tabla clientes en un ResultSet
String sql = "SELECT * FROM clientes";
ResultSet rs = stmt.executeQuery(sql);

// Desplazamos el cursor al tercer registro
rs.absolute(3)
rs.deleteRow();
```

Como ves, este apartado no usa las sentencias parametrizadas. Investiga cómo lanzarías directamente un DELETE al JDBC.

7. SENTENCIAS SQL CON JDBC

Para estos ejemplos supondremos que ya existe una conexión (con) y un Statement (stmt). **Crearemos** una tabla ALUMNOS en nuestra base de datos y añadiremos datos a la misma. La sentencia para la creación de la tabla será la siguiente:

```
String st_crea = "CREATE TABLE ALUMNOS (  
                exp INTEGER,  
                nombre VARCHAR(32),  
                nota VARCHAR(3) default 'NP',  
                PRIMARY KEY (exp)  
                )";  
  
stmt.executeUpdate(st_crea);
```

Una vez creada la tabla podremos **insertar** datos en ella como se muestra a continuación:

```
String st_inserta = "INSERT INTO ALUMNOS(exp, nombre) VALUES(1285, 'Manu')";  
  
stmt.executeUpdate(st_inserta);
```

Cuando tengamos datos dentro de la tabla, podremos **modificarlos** utilizando para ello una sentencia UPDATE:

```
String st_actualiza = "UPDATE FROM ALUMNOS  
                     SET nota = 'NOT' WHERE exp = 1285";  
  
stmt.executeUpdate(st_actualiza);
```

Si queremos **eliminar** un registro de la tabla utilizaremos una sentencia DELETE como se muestra a continuación:

```
String st_borra = "DELETE FROM ALUMNOS WHERE exp = 1285";  
  
stmt.executeUpdate(st_borra);
```

Como hemos visto mediante el método **executeUpdate()** del objeto Statement podremos ejecutar cualquier sentencia SQL, que no sea una consulta, en la base de datos a la que nos hayamos conectado, pudiendo así realizar cualquier acción que nos permita SQL de manera sencilla.

```
int n = stmt.executeUpdate(sentencia);
```

Como resultado nos devolverá un entero que podremos consultar, y que nos dirá el número de registros a los que ha afectado la operación en caso de sentencias INSERT, UPDATE y DELETE. En el caso de una sentencia DDL (Lenguaje de definición de datos), como puede ser por ejemplo la creación de una tabla, como no afecta a los datos contenidos en la BD devolverá siempre 0.

8. PATRONES DE DISEÑO APLICADOS AL ACCESO A BASES DE DATOS

En la unidad anterior, de desarrollo de interfaces gráficas, introdujimos el concepto de patrón de diseño y expusimos varios ejemplos dejando el de Singleton para tratar en esta unidad.

A medida que las aplicaciones crecen en tamaño y complejidad, el código relacionado con el acceso a la base de datos puede volverse complicado y difícil de mantener. Para abordar estos desafíos existen los patrones de diseño, que son soluciones probadas y establecidas para problemas comunes en el desarrollo de software. Los patrones de diseño proporcionan un enfoque estructurado y reutilizable para diseñar sistemas que son flexibles, escalables y fáciles de mantener.

En este apartado exploraremos 2 de los patrones más utilizados para gestionar las conexiones a base de datos desde nuestra aplicación, sin entrar en el mapeo objeto-relacional. Estos patrones ofrecen soluciones a problemas como la gestión de conexiones, la abstracción de la capa de acceso a datos y la separación de preocupaciones en el diseño de la aplicación. Al comprender y aplicar estos patrones, los desarrolladores pueden mejorar la modularidad, la flexibilidad y la mantenibilidad de sus aplicaciones de bases de datos.

8.1 El patrón Singleton

Cuando desarrollamos una aplicación que necesita acceder a una base de datos, es frecuente querer asegurarnos de que **solo haya una conexión activa** con la base de datos en todo momento. Esto ayuda a evitar sobrecargar el servidor con demasiadas conexiones y a ahorrar recursos del sistema. Para lograr esto, diseñaremos una clase basada en el patrón llamado "Singleton", que garantiza que solo haya una instancia de una clase activa, y por tanto en este caso, una conexión a la base de datos en toda la aplicación.

Singleton se encarga de iniciar el controlador de la base de datos, establecer la conexión con la base de datos (por ejemplo, MySQL), y luego devuelve un objeto de tipo "Connection". Este objeto nos permite utilizar métodos como `createStatement()` o `prepareStatement()`, etc...

Hay varias formas de abordar la creación de una clase con Singleton, sin embargo, en este ejemplo vamos a mostrar una forma simple de aplicar el concepto. En primer lugar vamos a ver la clase, usando Singleton, encargada de crear y controlar a la conexión:

```
public class ConexionBD {
    private static Connection con = null;
    public static Connection getConnection(){
        try{
            if(con == null){
                String url = "jdbc:mysql://localhost/nombreBasedatos";
                String user = "usuario";
                String passwd = "password";
                con = DriverManager.getConnection(url,user,passwd);
                System.out.println("Conexión creada satisfactoriamente!");
            }
        }
        catch(SQLException ex){
            ex.printStackTrace();
        }
        return con;
    }
}
```

Una vez creada la clase, se podría hacer uso de ella mediante un ejemplo como el siguiente:

```
try {
    Connection con = ConexionBD.getConnection();
    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery("SELECT * from clientes");
    // Resto del código
} catch (SQLException ex) {
    //gestión de excepción
}
```

Este es sólo un ejemplo muy simple de uso, se puede modificar y adaptar a las necesidades del programa que estemos desarrollando. Lo que tendremos que tener en cuenta que cumplir el precepto básico de Singleton (asegurar que hay sólo una instancia de la conexión a base de datos, o del objeto que se desee controlar).

8.2 El patrón DAO (Data Access Object)

El patrón “Data Access Object” (DAO) permite separar la lógica de acceso a datos de la lógica de negocio, de tal forma que el DAO encapsula y aísla el proceso de acceso a datos del resto de la aplicación.

Cuando hablamos de las interfaces gráficas introducimos la arquitectura Modelo Vista Controlador, que trataba de separar la parte de interacción con el usuario de la lógica del programa, para poder utilizar la parte lógica en otros entornos y para facilitar el mantenimiento.

Esto es algo similar, pero en relación con el acceso a la base de datos. Así, sería muy fácil cambiar de gestor e incluso de tipo de base de datos (BD orientada a objetos, BD NoSQL, etc...) ya que el resto de la aplicación no se vería afectada.

Por otra parte, en las actualizaciones y correcciones referente al acceso a la base datos, es más fácil localizar el código afectado. La idea sería entonces tener una clase que agrupase todas estas operaciones.

Supongamos que tenemos una aplicación que gestiona una clase llamada **Empleado**:

```
public class Empleado {
    private int id;
    private String nombre;
    private LocalDate fechaNacimiento;
    private String categoria;
    private float salario;
    // + constructor con todos los atributos, getters y setters
}
```

y que tenemos los datos de esa clase en una base de datos, concretamente en una tabla llamada Empleado.

El patrón DAO nos dice que debemos hacer una clase que englobe todos los accesos a Empleado con la base de datos, tanto de consulta como de modificación.

1) La conexión la podríamos establecer en el constructor. Vamos a seguir el patrón Singleton visto previamente:

```

public class DaoEmpleado {
    private static DaoEmpleado instance = null;
    private Connection con = null;

    private DaoEmpleado() throws SQLException, IOException {
        String JDBC_URL = "jdbc:mysql://localhost:3306/empresa";
        if (con == null) {
            con = DriverManager.getConnection(JDBC_URL, "user", "pass");
        }
    }

    public static DaoEmpleado getInstance() throws SQLException {
        if (instance == null)
            instance = new DaoEmpleado();
        else
            return instance;
    }
    . . .
}

```

2) Ahora podríamos hacer métodos para la inserción, búsqueda, o cualquier otra funcionalidad que precisase nuestra aplicación:

```

public int insert(Empleado e) throws SQLException {
    int cantFilas;
    PreparedStatement ps = con.prepareStatement("INSERT INTO empleado (id, nombre, fechaNacimiento, categoria, salario) VALUES( ?, ?, ?, ?, ?)");
    ps.setInt(1, e.getId());
    ps.setString(2, e.getNombre());
    ps.setDate(3, Date.valueOf(e.getFechaNacimiento()));
    ps.setString(4, e.getCategoria());
    ps.setFloat(5, e.getSalario());
    cantFilas = ps.executeUpdate();
    ps.close();
    return cantFilas;
}

public List<Empleado> findAll() throws SQLException {
    PreparedStatement ps = con.prepareStatement("SELECT * FROM empleado");
    ResultSet rs = ps.executeQuery();
    List<Empleado> result = new ArrayList<>();

    while (rs.next()) {
        result.add(new Empleado(rs.getInt("id"), rs.getString("nombre"),
            rs.getDate("fechaNacimiento").toLocalDate(),
            rs.getString("categoria"),
            rs.getFloat("salario")));
    }

    rs.close();
    ps.close();
    return result;
}

```


3) Ahora, desde las clases de la aplicación llamaremos a los métodos de la clase DAO, primero estableciendo la conexión y luego ejecutando la operación deseada. Los métodos del objeto DAO lanzan excepciones, por lo que los programas y clases que los llamen deberán capturarlas. Por ejemplo:

```
try {  
    DaoEmpleado daoEmp = DaoEmpleado.getInstance();  
    daoEmp.insert(new Empleado(12, "Ana Pérez", "31/12/1999", "Peón", 1400f));  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

9. MAPEO OBJETO-RELACIONAL

Debemos empezar hablando del “desfase” objeto-relacional, el cual surge cuando en el desarrollo de una aplicación con un lenguaje orientado a objetos se hace uso de una base de datos relacional para almacenar la información que, en la aplicación, va ubicada en clases y objetos que no han de tener una correspondencia exacta con el modelo de base de datos. Hay que tener en cuenta que esta situación se da porque tanto los lenguajes orientados a objetos como las bases de datos relacionales están ampliamente extendidas.

En cuanto al desfase, ocurre que en nuestra aplicación Java (como ejemplo de lenguaje Orientado a Objetos) tendremos, por ejemplo, la definición de una clase cualquiera con sus atributos y métodos:

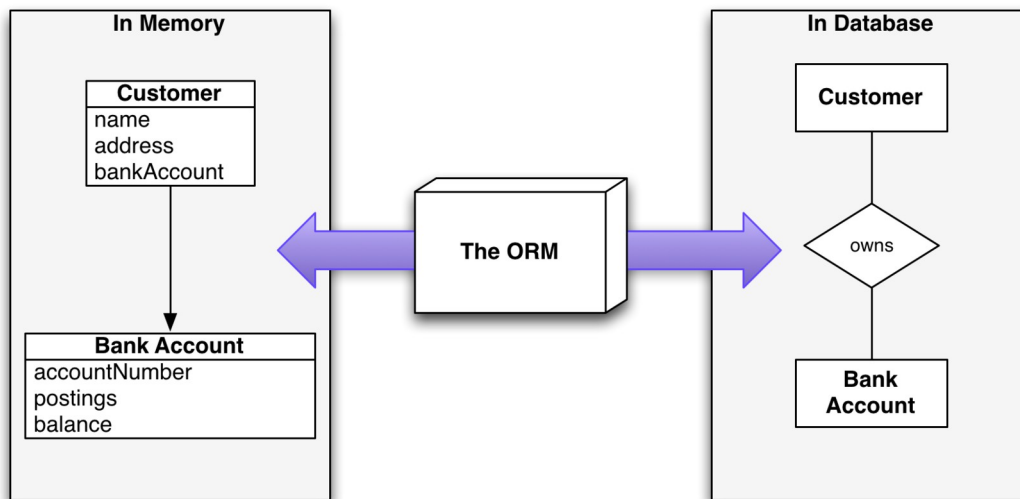
```
public class Personaje {  
    private int id;  
    private String nombre;  
    private String descripcion;  
    private int vida;  
    private int ataque;  
  
    public Personaje(. . .) {  
        . . .  
    }  
}
```

Mientras que en la base de datos tendremos una tabla cuyos campos corresponderán directamente (o no) con los atributos que hayamos definido anteriormente en esa clase. Puesto que son estructuras que no tienen nada que ver entre ellas, tendremos que hacer el mapeo manualmente, haciendo coincidir (a través de los getters o setters) cada uno de los atributos con cada uno de los campos (y viceversa) cada vez que queramos leer o escribir un objeto desde y hacia la base de datos, respectivamente. Cuestión que se podrá hacer mediante instrucciones con el JDBC.

```
CREATE TABLE characters (  
    id INT PRIMARY KEY AUTO_INCREMENT;  
    name VARCHAR(50) NOT NULL,  
    description VARCHAR(50),  
    life INT DEFAULT 10,  
    attack_power INT DEFAULT 10;  
);
```

Eso hace que tengamos que estar continuamente descomponiendo los objetos para escribir la sentencia SQL para insertar, modificar o eliminar, o bien recomponer todos los atributos para formar el objeto cuando leamos algo de la base de datos. El ejemplo anterior es “sencillo” ya que hemos usado una clase cuya estructura coincide con la tabla de la base de datos, pero esto no siempre será tan sencillo y ahí es donde tendremos más problemas.

El concepto de mapeo objeto-relacional (ORM) se puede gestionar manualmente (JDBC) o mediante frameworks que nos facilitan esa labor (Hibernate). Veamos un esquema:



Volviendo al ejemplo del Personaje, si trabajamos directamente con JDBC, entonces el mapeo objeto-relacional lo tendremos que hacer manualmente descomponiendo el objeto para construir la sentencia INSERT:

```
String sql = "INSERT INTO personajes (nombre, descripcion, vida, ataque)" +  
    " VALUES (?, ?, ?, ?)";  
PreparedStatement pStm = conexion.prepareStatement(sql);  
pStm.setString(1, personaje.getNombre());  
pStm.setString(2, personaje.getDescripcion());  
pStm.setInt(3, personaje.getVida());  
pStm.setInt(4, personaje.getAtaque());  
pStm.executeUpdate();  
  
if (pStm != null)  
    pStm.close();
```

Sin embargo, si contamos con un framework como **Hibernate**, esta misma operación se traduce en unas pocas líneas de código en las que podemos trabajar directamente con el objeto Java, puesto que el framework realiza el mapeo en función de las anotaciones que hemos implementado a la hora de definir la clase, y que le indican a éste con que tabla y campos de la misma se corresponde la clase y sus atributos, respectivamente; olvidando la necesidad de construir manualmente las sentencias SQL.

```
@Entity  
@Table(name="personajes")  
public class Personaje {  
    @Id // Marca el campo como la clave de la tabla  
    @GeneratedValue(strategy = IDENTITY)  
    @Column(name="id")  
    private int id;  
  
    @Column(name="name")  
    private String nombre;
```

```

@Column(name="description")
private String descripcion;

@Column(name="life")
private int vida;

@Column(name="attack_power")
private int ataque;

public Personaje(. . .) {
    . . .
}
}

```

Partiendo de esa definición de la clase; podemos simplemente establecer una sesión con la base de Datos y enviarle el objeto, en este caso invocando al método **save**, que se encarga de registrarlo en la base de datos donde convenga según sus propias anotaciones, y no teniendo que usar JDBC ni instrucciones SQL:

```

sesion = HibernateUtil.getCurrentSession();
sesion.beginTransaction();
sesion.save(personaje);
sesion.getTransaction().commit();
sesion.close();

```

El uso y configuración de Hibernate excede los objetivos de este curso, pero sí es importante al menos entender el concepto y la necesidad del mapeo objeto-relacional.

Finalmente, veamos un ejemplo un poco más sofisticado en el que se aprecia mejor el desfase objeto-relacional mediante una relación muchos a muchos entre las entidades “Usuario” y “Rol”. En el modelo de objetos sólo tenemos 2 clases, la clase “User” y la clase “Role”, sin embargo, en el modelo de la base de datos hay 3 tablas; por tanto, tendremos que construir algún tipo de correspondencia entre los datos de las 2 clases y las 3 tablas. Esto se puede hacer manualmente con sentencias SQL, pero se complica bastante, ahí es donde entra en juego nuestro ORM como Hibernate.

En la siguiente página se puede apreciar el diagrama de lo mencionado anteriormente.

Many To Many Relationship in ORM

```
@Entity
public class User {

    @Id
    private Integer userId;
    private String userName;
    private String userPwd;
```

```
    @ManyToMany
    @JoinTable(name = "user_role",
        joinColumns = @JoinColumn(name = "user_fk"),
        inverseJoinColumns = @JoinColumn(name = "role_fk")
    )
```

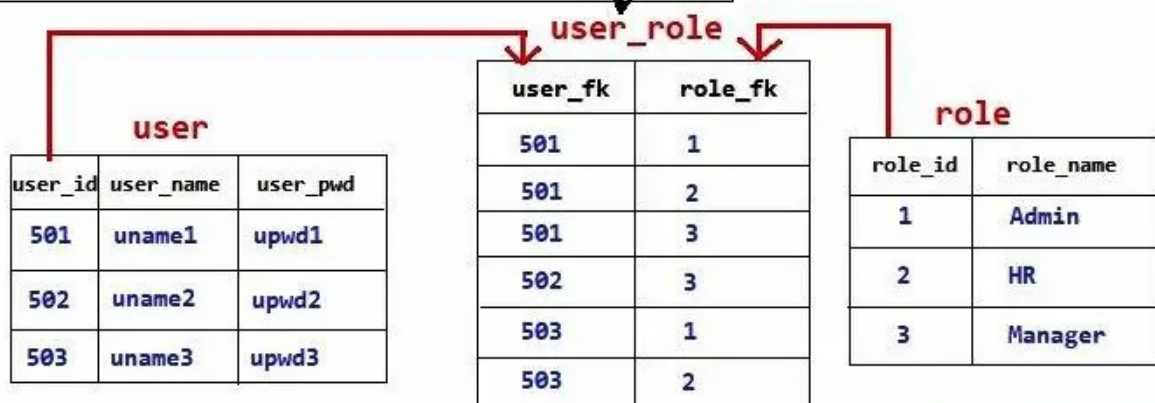
```
    private List<Role> roles;
```

```
}
```

User  Role

```
@Entity
public class Role {

    @Id
    private Integer roleId;
    private String roleName;
}
```



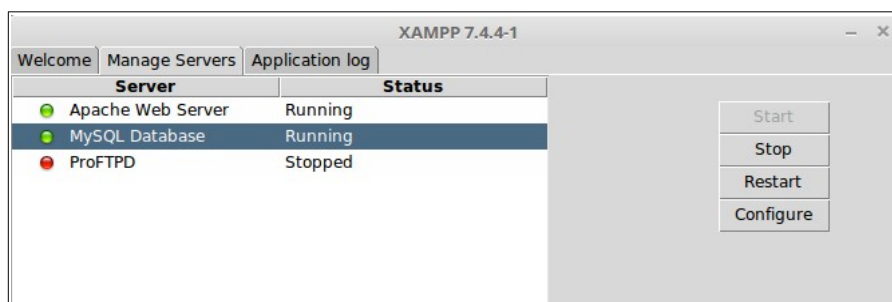
@javatechonline.com

10. EJEMPLO: PROGRAMA GESTOR DE CLIENTES CON XAMPP

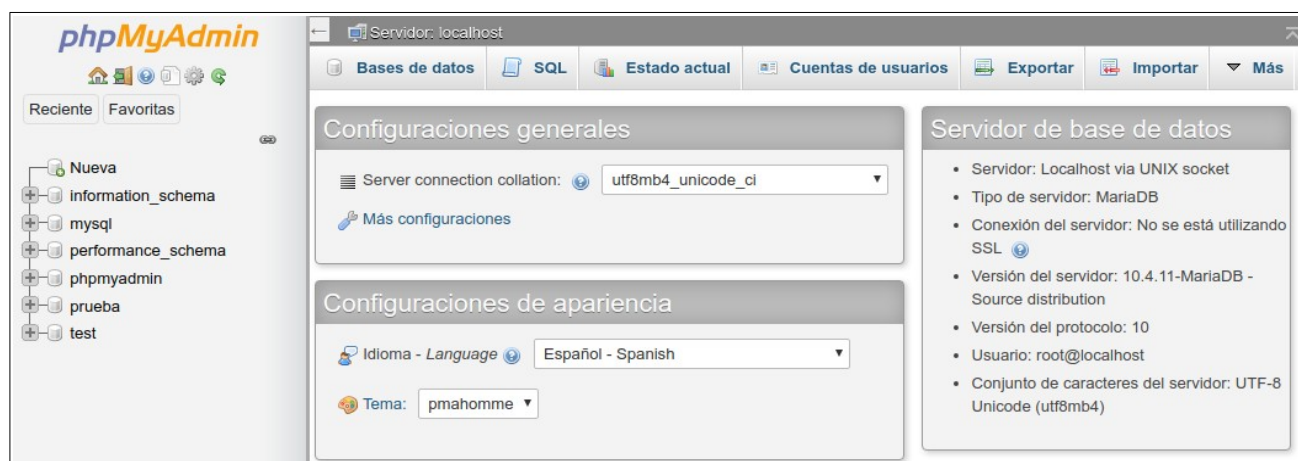
Veamos un ejemplo de un programa de gestión de clientes de una tienda, con interfaz gráfica de usuario y acceso a base de datos MySQL. Este programa permitirá consultar y modificar la información de los clientes así como darlos de alta y de baja. El código del proyecto y la base de datos pueden encontrarse en el aula virtual.

En la base de datos tendremos una sola tabla con los campos *id*, *nombre* y *dirección*. Cabe destacar que el campo *id* es autoincremental, es decir que su valor se incrementará al insertar un registro en la tabla, por lo que al crear nuevos registros no es necesario darle un valor.

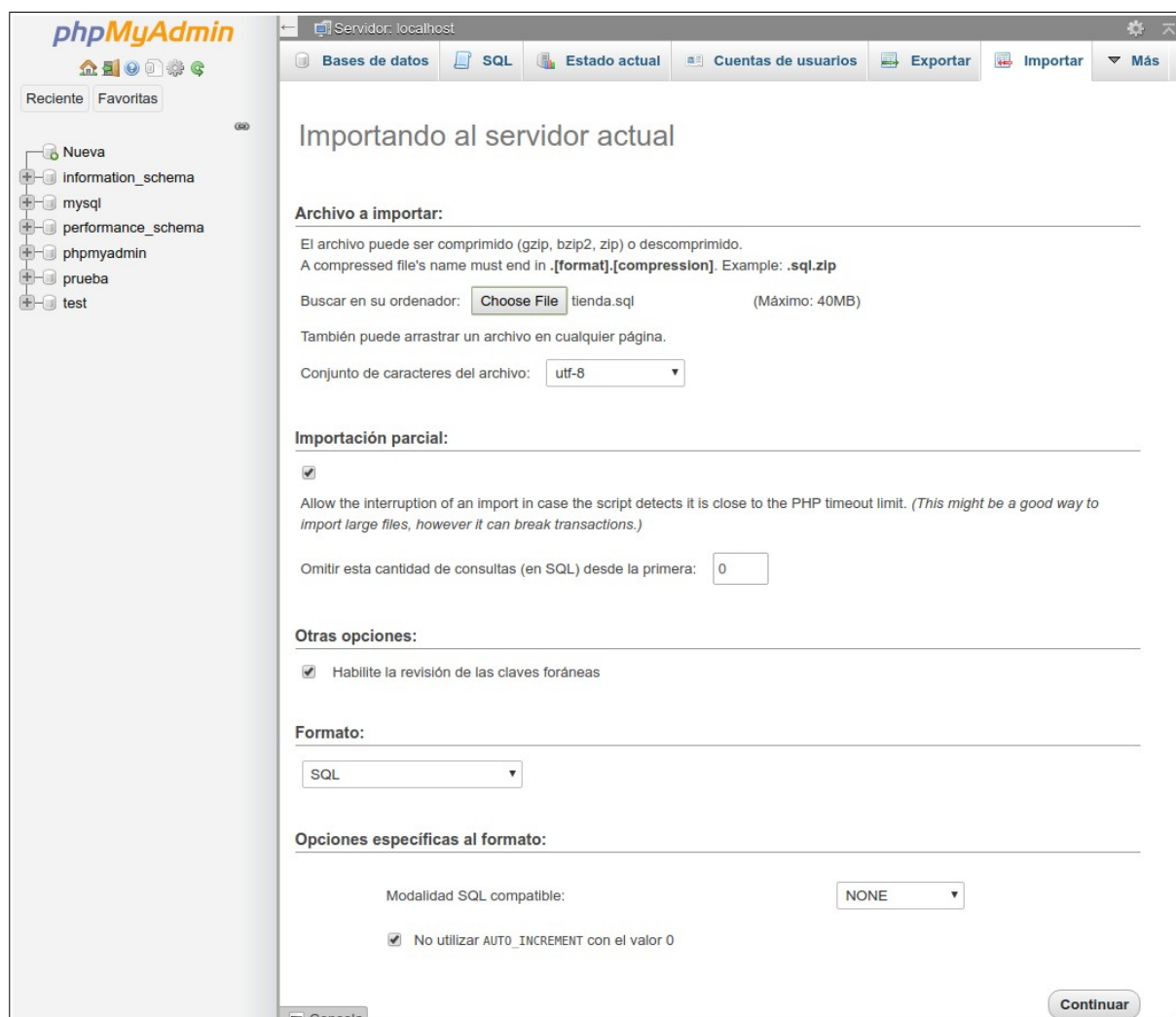
Primero necesitamos importar la base de datos a nuestro servidor MySQL. Abrimos el panel de control XAMPP y nos aseguramos de que *Apache* y *MySQL* están funcionando. En caso contrario los iniciamos.



Abrimos un navegador web, vamos a <http://localhost> y accedemos a phpMyAdmin. También podemos acceder directamente con <http://localhost/phpmyadmin/>. Esta aplicación web alojada en nuestro servidor Apache nos permite interactuar con el servidor MySQL de una forma sencilla.

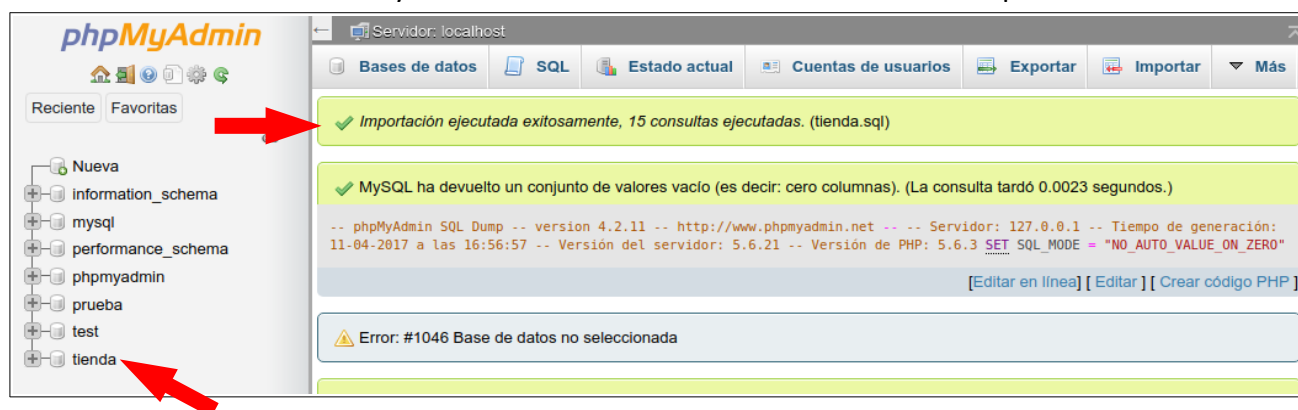


A continuación hacemos clic en 'Importar' y seleccionamos el archivo 'tienda.sql' que podéis descargar del aula virtual. Este archivo contiene las sentencias sql para crear la base de datos que necesitamos.



The screenshot shows the phpMyAdmin interface for importing a file. The left sidebar lists databases: Nueva, information_schema, mysql, performance_schema, phpmyadmin, prueba, and test. The main panel is titled 'Importando al servidor actual'. It includes sections for 'Archivo a importar' (with a 'Choose File' button and 'utf-8' encoding), 'Importación parcial' (with a checkbox for interruption), 'Otras opciones' (with a checkbox for foreign key checks), 'Formato' (set to 'SQL'), and 'Opciones específicas al formato' (with a checkbox for 'No utilizar AUTO_INCREMENT'). A 'Continuar' button is at the bottom right.

Hacemos clic en 'Continuar' y si todo va bien la base de datos 'tienda' se importará correctamente.



This screenshot shows the phpMyAdmin interface after the import. The left sidebar has a red arrow pointing to the 'tienda' database. The main panel displays a green success message: 'Importación ejecutada exitosamente, 15 consultas ejecutadas. (tienda.sql)'. Below it, another green message states: 'MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0.0023 segundos.)'. A blue box contains technical details about the phpMyAdmin SQL Dump. At the bottom, a yellow error message reads: 'Error: #1046 Base de datos no seleccionada'.

Ya tenemos la base de datos en nuestro servidor MySQL.

El siguiente paso es desarrollar la aplicación. Debemos crear un nuevo proyecto Java e importar el JDBC connector como una librería JAR tal y como se ha explicado al principio de esta unidad.

A continuación creamos un JFrame llamado **VentanaClientes.java** y diseñamos la interfaz gráfica. Por ejemplo así:



El código relacionado con el acceso a la base de datos estará en una clase distinta llamada **BDClientes.java**. Esta clase contendrá varios métodos públicos para las distintas operaciones que necesitaremos utilizar desde la interfaz gráfica: conectar a la base de datos, consultar el listado de clientes, así como insertar, actualizar y borrar clientes.

Es decir, la clase VentanaClientes contendrá el código de la interfaz gráfica (JFrame) y utilizará la clase BDClientes para las operaciones de acceso a la base de datos. Esta división de 'responsabilidades' facilita el desarrollo y hace que nuestro código sea más modular acercándonos a la arquitectura modelo-vista-controlador.

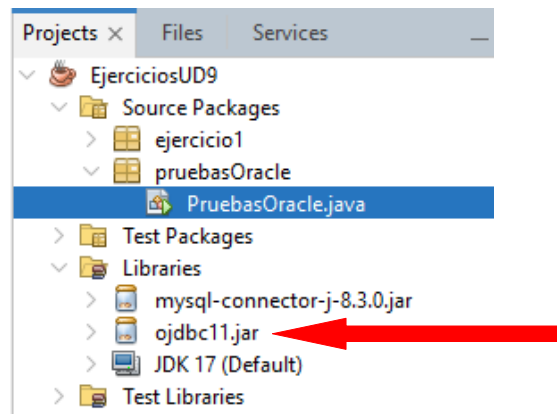
ANEXO I: CONECTAR A BASES DE DATOS ORACLE XE

El driver recomendado se puede descargar desde:

<https://www.oracle.com/es/database/technologies/appdev/jdbc-downloads.html>

Oracle Database 23c (23.3.0.23.09) JDBC Driver & UCP Downloads - Free Release		
Supports Oracle Database versions - 23c, 21c, and 19c Refer to JDBC-UCP-ReleaseNotes-23c.txt and B		
Note: The Oracle JDBC 23.3 drivers are supported for production use with BaseDB only. The 23.3 release drivers are not supp		
Name	Download	JDK Supported
Oracle JDBC driver	 ojdbc11.jar	Implements JDBC 4.3 spec and certified with JDK11, JDK17, JDK19, and JDK21
Oracle JDBC Driver	 ojdbc8.jar	Implements JDBC 4.2 spec and certified with JDK8 and JDK11

En rojo están marcados los elementos en que hay que fijarse (versiones de base de datos soportadas, y versiones del JDK soportadas). Se denomina: “Oracle JDBC driver”, el fichero concreto es: “ojdbc11.jar”. Una vez descargado lo colocaremos en nuestro proyecto tal y como se ve en la siguiente imagen:



Recuerda importar en tu proyecto las clases necesarias del paquete `java.sql.*`

La URL de conexión debe ser la siguiente:

```
jdbc:oracle:thin:@localhost:1521:xe
```

Al final aparece el SID, o esquema, que tendrás que cambiar si no usas el esquema por defecto “xe”.

Usuario: SYSTEM (o el que estés usando actualmente).

Contraseña: la que se puso en la instalación o en el nuevo usuario.

Por último, recuerda que, al estar usando Oracle, es posible que necesites ejecutar algún commit en código Java.